**BỘ KHOA HỌC VÀ CÔNG NGHỆ**
**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**
**KHOA CÔNG NGHỆ THÔNG TIN 1**

-----□□ □ □□-----



# BÁO CÁO BÀI TẬP LỚN
# HỌC PHẦN: LẬP TRÌNH MẠNG

# ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GỌI MÓN SERVIO THỜI GIAN THỰC QUA WEBSOCKET/STOMP

**Giảng viên** : MSc. Ngô Văn Quyền
**Họ và tên sinh viên** : Lê Đình Chuyên – B23DCCN100
Vũ Ngọc Diệp – B23DCCN156
Trần Thị Ánh Dương – B23DCCN230
Bùi Huy Hoàng – B23DCCN324
**Chuyên ngành** : Công nghệ phần mềm
**Khoa** : Công nghệ thông tin 1

Hà Nội, năm 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

-----□□ □ □□-----




# BÁO CÁO BÀI TẬP LỚN
# HỌC PHẦN: LẬP TRÌNH MẠNG

# ĐỀ TÀI: HỆ THỐNG GỌI MÓN SERVIO ĐỒNG BỘ ĐƠN HÀNG THỜI GIAN THỰC QUA WEBSOCKET/STOMP

**Giảng viên** : MSc. Ngô Văn Quyền

**Họ và tên sinh viên** : Lê Đình Chuyên – B23DCCN100

Vũ Ngọc Diệp – B23DCCN156

Trần Thị Ánh Dương – B23DCCN230

Bùi Huy Hoàng – B23DCCN324

**Chuyên ngành** : Công nghệ phần mềm

**Khoa** : Công nghệ thông tin 1

Hà Nội, năm 2026

# LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành đến ThS. Ngô Văn Quyền, giảng viên học phần Lập trình mạng, đã tận tình giảng dạy, truyền đạt kiến thức và định hướng cho chúng em trong suốt quá trình học tập và thực hiện bài tập lớn.

Thông qua đề tài "Hệ thống gọi món Servio đồng bộ đơn hàng thời gian thực qua WebSocket/STOMP", nhóm đã có cơ hội vận dụng kiến thức lý thuyết vào thực tiễn, đồng thời nâng cao kỹ năng lập trình, nghiên cứu và làm việc nhóm.

Mặc dù đã cố gắng hoàn thiện đề tài, song do thời gian và kinh nghiệm còn hạn chế, báo cáo khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những ý kiến đóng góp của thầy để tiếp tục hoàn thiện sản phẩm.

Nhóm chúng em xin chân thành cảm ơn!

# MỤC LỤC

**LỜI CẢM ƠN** ..... **1**
**TÓM TẮT** ..... **4**
**I. GIỚI THIỆU** ..... **4**
**II. NGHIÊN CỨU LIÊN QUAN** ..... **6**
2.1. Tổng quan các hệ thống liên quan ..... 6
2.2. Đánh giá các hệ thống liên quan ..... 6
2.3. So sánh các hệ thống liên quan ..... 7
2.4. Hạn chế của các hệ thống liên quan và định hướng phát triển Servio ..... 7
**III. PHƯƠNG PHÁP NGHIÊN CỨU** ..... **8**
3.1. Xác định bài toán ..... 8
3.2. Kiến trúc hệ thống đề xuất ..... 8
3.3. Thiết kế giao tiếp Client và Server ..... 9
3.4. Cơ chế giao nhận sự kiện tin cậy ..... 10
3.5. Kiểm soát yêu cầu lặp lại và cập nhật đồng thời ..... 12
3.6. Quy trình xử lý đơn hàng ..... 13
3.7. Trợ lý AI gợi ý món theo ngữ cảnh thời gian thực ..... 13
3.8. Phương pháp đánh giá ..... 13
**IV. THỰC NGHIỆM VÀ KẾT QUẢ** ..... **14**
4.1. Môi trường và phương pháp thực nghiệm ..... 14
4.2. Kết quả kiểm thử chức năng ..... 14
4.3. Phương pháp đo đạc thực nghiệm đối chứng (E1 – E6) ..... 15
4.4. Kết quả thực nghiệm đối chứng ..... 15
**V. THẢO LUẬN** ..... **16**
5.1. Đánh giá hiệu quả đồng bộ thời gian thực và khôi phục sự kiện ..... 16
5.2. Đánh giá khả năng kiểm soát tương tranh, yêu cầu lặp và chống quá tải ..... 16
5.3. Đối chiếu với các hệ thống liên quan và hạn chế của đề tài ..... 17
**VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN** ..... **18**
6.1. Kết luận ..... 18
6.2. Hướng phát triển ..... 18
**TÀI LIỆU THAM KHẢO** ..... **20**
**PHỤ LỤC** ..... **21**

Phụ lục A. Hình ảnh giao diện thực tế của chương trình.....21
Phụ lục B. Hướng dẫn khởi động hệ thống, tài khoản mẫu và chạy kiểm thử.....22

## TÓM TẮT

Trong hệ thống gọi món tại nhà hàng, việc chuyển đơn từ khách hàng đến bếp và cập nhật trạng thái xử lý cần được thực hiện nhanh, đồng thời bảo đảm dữ liệu nhất quán giữa các thiết bị. Nếu chỉ sử dụng HTTP polling, thiết bị phải liên tục gửi yêu cầu kiểm tra đơn hàng. Trong khi đó, cách đẩy thông báo qua WebSocket thông thường có thể bỏ lỡ sự kiện khi thiết bị mất kết nối.

Đề tài xây dựng Servio, hệ thống gọi món tại bàn bằng mã QR, sử dụng WebSocket/STOMP để đồng bộ đơn hàng giữa khách hàng, bếp và thu ngân. Hệ thống được phát triển với React, Spring Boot và MySQL, kết hợp HTTP/REST cho các thao tác nghiệp vụ. Để xử lý các tình huống mạng bất thường, nhóm triển khai cơ chế xác nhận và phát lại sự kiện thông qua ACK, RESUME và REPLAY. Ngoài ra, Idempotency-Key và khóa lạc quan được sử dụng để hạn chế tạo đơn trùng và xung đột cập nhật.

Trong môi trường kiểm thử sử dụng H2 chạy trong bộ nhớ, kết quả thực nghiệm cho thấy độ trễ trung vị khi bếp nhận đơn giảm từ 1.341,68 ms với polling xuống 7,71 ms với WebSocket/STOMP. Ở kịch bản ngắt kết nối, phương án đẩy thông báo thông thường bỏ lỡ 31/200 đơn, trong khi cơ chế có khôi phục sự kiện không ghi nhận đơn bị mất. Bộ kiểm thử cũng đạt 29/29 ca tích hợp và 212/212 ca đầu cuối.

Kết quả bước đầu cho thấy giải pháp phù hợp với yêu cầu đồng bộ đơn hàng của Servio. Tuy nhiên, nhóm cần tiếp tục đánh giá trên cơ sở dữ liệu MySQL và môi trường mạng thực tế trước khi triển khai vận hành.



Từ khóa: WebSocket, STOMP, gọi món bằng mã QR, đồng bộ thời gian thực, khôi phục sự kiện.


## I. GIỚI THIỆU

Trong những năm gần đây, việc ứng dụng công nghệ vào hoạt động phục vụ tại nhà hàng ngày càng phổ biến. Một trong những hình thức được sử dụng là gọi món thông qua mã QR, cho phép khách hàng truy cập thực đơn và gửi yêu cầu đặt món trực tiếp từ điện thoại. Cách tiếp cận này giúp giảm bớt các thao tác ghi nhận đơn hàng thủ công và tạo điều kiện để nhà hàng quản lý thông tin tập trung hơn. Tuy nhiên, khi nhiều khách hàng cùng đặt món, hiệu quả của hệ thống không chỉ phụ thuộc vào khả năng tiếp nhận yêu cầu mà còn ở tốc độ và độ chính xác khi chuyển thông tin giữa các bộ phận.

Trong quy trình gọi món, một đơn hàng có thể trải qua nhiều trạng thái từ lúc được tạo, tiếp nhận, chế biến cho đến khi hoàn thành và thanh toán. Những thay đổi này cần được phản ánh trên các thiết bị của khách hàng, bếp và thu ngân. Nếu thông tin cập nhật chậm, bộ phận bếp có thể không nhận được đơn mới kịp thời. Ngược lại, nếu trạng thái giữa các thiết bị không thống nhất, nhân viên có thể xử lý dựa trên dữ liệu đã cũ. Đây là những tình huống cần được xem xét khi thiết kế cơ chế giao tiếp mạng cho hệ thống.

Với các ứng dụng web thông thường, HTTP polling là một cách triển khai tương đối đơn giản. Thiết bị gửi yêu cầu đến máy chủ theo chu kỳ để kiểm tra dữ liệu mới. Tuy nhiên, thời gian nhận được thay đổi phụ thuộc vào khoảng cách giữa hai lần truy vấn. Việc rút ngắn chu kỳ giúp giảm thời gian chờ nhưng đồng thời làm tăng số lượng yêu cầu gửi đến máy chủ, kể cả khi không có đơn hàng mới. WebSocket cung cấp một hướng tiếp cận khác bằng cách duy trì kết nối hai chiều, cho phép máy chủ chủ động gửi sự kiện ngay khi dữ liệu thay đổi.

Dù có lợi thế về tốc độ cập nhật, WebSocket không tự giải quyết toàn bộ vấn đề đồng bộ dữ liệu. Khi một thiết bị mất kết nối, các thông báo được gửi trong khoảng thời gian đó có thể không đến được thiết bị. Sau khi kết nối lại, nếu chỉ tiếp tục nhận các thông báo mới, thiết bị vẫn có nguy cơ thiếu những thay đổi đã xảy ra trước đó. Bên cạnh đó, kết nối không ổn định có thể khiến người dùng gửi lại cùng một yêu cầu đặt món. Việc nhiều nhân viên cùng cập nhật trạng thái một đơn hàng cũng có thể dẫn đến xung đột nếu hệ thống không kiểm soát được thứ tự và phiên bản dữ liệu.

Như vậy, bài toán đặt ra không đơn thuần là lựa chọn giữa HTTP và WebSocket mà là xây dựng một cơ chế giao tiếp vừa có độ trễ thấp, vừa hỗ trợ khôi phục thông tin sau gián đoạn và duy trì tính nhất quán khi có nhiều thao tác đồng thời. Các yêu cầu này cần được giải quyết ở cả tầng truyền sự kiện và tầng xử lý nghiệp vụ. Việc chỉ gửi thông báo theo thời gian thực là chưa đủ để bảo đảm trạng thái đơn hàng luôn được đồng bộ trong những tình huống bất thường.

Từ bài toán trên, nhóm tập trung nghiên cứu cách kết hợp WebSocket/STOMP với các cơ chế xác nhận, phát lại sự kiện và kiểm soát cập nhật dữ liệu. Trong phạm vi đề tài, nhóm lựa chọn khảo sát các hệ thống gọi món mã nguồn mở để đối chiếu cách tổ chức nghiệp vụ và giao tiếp giữa các thành phần. Việc phân tích những hệ thống này là cơ sở để xác định các yêu cầu kỹ thuật mà Servio cần giải quyết; các nhận xét và giới hạn cụ thể sẽ được trình bày trong phần nghiên cứu liên quan.

Nhóm xây dựng Servio – Hệ thống gọi món đồng bộ đơn hàng thời gian thực qua WebSocket/STOMP. Khách hàng quét mã QR tại bàn để xem thực đơn và đặt món. Máy chủ tiếp nhận, xử lý yêu cầu và gửi sự kiện đến bộ phận bếp, thu ngân hoặc các thiết bị liên quan. Hệ thống sử dụng HTTP/REST cho các thao tác nghiệp vụ, kết hợp WebSocket/STOMP để phân phối thông báo thay đổi trạng thái.

Các đóng góp kỹ thuật chính của đề tài gồm:

- Thiết kế cơ chế đồng bộ đơn hàng: Kết hợp HTTP/REST và WebSocket/STOMP, phân chia trách nhiệm giữa xử lý yêu cầu nghiệp vụ và truyền sự kiện đến các thiết bị.
- Hỗ trợ khôi phục sự kiện khi mất kết nối: Sử dụng số thứ tự sự kiện cùng cơ chế ACK, RESUME và REPLAY để thiết bị có thể yêu cầu nhận lại những thay đổi bị bỏ lỡ.
- Kiểm soát yêu cầu trùng và cập nhật đồng thời: Áp dụng Idempotency-Key để hạn chế tạo đơn hàng lặp và khóa lạc quan để phát hiện xung đột khi nhiều thiết bị cùng thay đổi trạng thái.

- Xây dựng thực nghiệm đối chứng: So sánh độ trễ giữa HTTP polling và WebSocket/STOMP; đánh giá khả năng khôi phục sự kiện, xử lý yêu cầu lặp và cập nhật đồng thời thông qua các kịch bản kiểm thử.

Đề tài được triển khai dưới dạng ứng dụng web với React, Spring Boot và MySQL. Các thử nghiệm được thực hiện trong môi trường kiểm thử của nhóm nhằm đánh giá các cơ chế giao tiếp và xử lý dữ liệu đã xây dựng. Kết quả thực nghiệm là cơ sở để nhận xét về hiệu quả của giải pháp trong phạm vi thử nghiệm, đồng thời xác định những vấn đề cần tiếp tục đánh giá trước khi triển khai thực tế.

# II. NGHIÊN CỨU LIÊN QUAN

## 2.1. Tổng quan các hệ thống liên quan

Các hệ thống gọi món trực tuyến hiện nay không chỉ hỗ trợ thực đơn điện tử mà còn cho phép đặt món và cập nhật trạng thái đơn hàng. Để tìm hiểu những cách triển khai đã có, nhóm khảo sát ba dự án mã nguồn mở [1]–[3], tập trung vào phương thức giao tiếp, khả năng đồng bộ dữ liệu và xử lý gián đoạn kết nối.

## 2.2. Đánh giá các hệ thống liên quan

### 2.2.1. QR Code Generator and Restaurant Menu Website

Dự án của jshi98 [1] sử dụng Python và Flask để tạo mã QR liên kết với thực đơn nhà hàng. Khách hàng có thể quét mã để truy cập danh sách món ăn trên giao diện web tương thích với nhiều thiết bị. Ưu điểm của dự án là quy trình truy cập thực đơn đơn giản và không yêu cầu khách hàng cài đặt ứng dụng. Tuy nhiên, tài liệu dự án chủ yếu mô tả chức năng tạo mã QR và hiển thị thực đơn, chưa trình bày quy trình xử lý đơn hàng giữa khách hàng và bếp. Do đó, dự án phù hợp để tham khảo cách tổ chức thực đơn điện tử nhưng chưa cung cấp đủ thông tin để đánh giá cơ chế đồng bộ đơn hàng.

### 2.2.2. QR Ordering System

Dự án của Codewithaffan [2] sử dụng Next.js, MongoDB và Socket.IO để xây dựng hệ thống gọi món bằng mã QR. Khách hàng đặt món tại bàn, trong khi bộ phận quản lý nhận thông báo đơn mới và cập nhật trạng thái xử lý. Hệ thống sử dụng Socket.IO để truyền sự kiện và truy vấn REST để đồng bộ lại dữ liệu sau khi kết nối được khôi phục. Ngoài ra, dự án áp dụng cập nhật có điều kiện nhằm ngăn hai quản trị viên thay đổi cùng một trạng thái theo cách xung đột. Cách triển khai này giải quyết được nhiều yêu cầu thực tế của hệ thống gọi món. Tuy nhiên, tài liệu dự án không mô tả cơ chế phát lại từng sự kiện nghiệp vụ dựa trên số thứ tự đã xác nhận; việc khôi phục được thực hiện bằng cách truy vấn lại dữ liệu hiện tại.

### 2.2.3. Real-Time Restaurant Table Ordering System

Dự án của ar363 [3] được xây dựng bằng Next.js, Django và WebSocket, hỗ trợ đặt món và cập nhật trạng thái giữa khách hàng với bộ phận bếp. Điểm nổi bật là khả năng đồng bộ giỏ hàng giữa nhiều thiết bị, cho phép những người cùng tham gia đặt món theo dõi các thay đổi gần như tức thời. Hệ thống cũng cung cấp giao diện bếp để tiếp

nhận đơn và cập nhật tiến độ chế biến. Đây là hướng triển khai phù hợp với các nhà hàng có nhiều thiết bị cùng truy cập. Tuy nhiên, tài liệu công khai chưa trình bày rõ cách xác định và phát lại những sự kiện đơn hàng bị bỏ lỡ khi client mất kết nối, nên chưa đủ cơ sở để đánh giá khả năng khôi phục sự kiện của hệ thống.

## 2.3. So sánh các hệ thống liên quan

Bảng 2.1. So sánh các hệ thống được khảo sát

| Tiêu chí | QR Menu [1] | QR Ordering [2] | Restaurant Live Ordering [3] |
|---|---|---|---|
| Công nghệ chính | Flask, Python | Next.js, MongoDB, Socket.IO | Next.js, Django, WebSocket |
| Phạm vi xử lý | Tạo QR, hiển thị thực đơn | Đặt món, quản lý đơn và trạng thái | Đặt món, đồng bộ giỏ hàng, quản lý đơn |
| Cập nhật thời gian thực | Không có | Socket.IO | WebSocket |
| Xử lý khi kết nối lại | Không có | Truy vấn REST để đồng bộ dữ liệu | Không có |
| Kiểm soát cập nhật đồng thời | Không có | Cập nhật có điều kiện | Không có |

Kết quả so sánh cho thấy các dự án có sự khác biệt về phạm vi và cách tổ chức giao tiếp. Dự án [1] tập trung vào thực đơn điện tử, trong khi [2] và [3] đã hỗ trợ cập nhật đơn hàng theo thời gian thực. Đáng chú ý, dự án [2] có phương án khôi phục dữ liệu sau khi kết nối lại và kiểm soát xung đột cập nhật. Vì vậy, việc sử dụng WebSocket hoặc xử lý mất kết nối không thể được xem là điểm mới riêng của Servio.

## 2.4. Hạn chế của các hệ thống liên quan và định hướng phát triển Servio

QR Menu [1] mới tập trung vào thực đơn điện tử, chưa có chức năng đồng bộ đơn hàng theo thời gian thực. QR Ordering [2] đã hỗ trợ cập nhật đơn hàng qua Socket.IO, đồng bộ lại dữ liệu khi kết nối được khôi phục và kiểm soát cập nhật đồng thời. Tuy nhiên, dự án chưa mô tả cơ chế phát lại các sự kiện bị bỏ lỡ. Restaurant Live Ordering [3] hỗ trợ cập nhật đơn hàng qua WebSocket nhưng chưa làm rõ cách xử lý mất kết nối và xung đột cập nhật giữa nhiều thiết bị. Bên cạnh đó, cả ba dự án đều chỉ cung cấp thực đơn tĩnh: khách hàng tự tìm và chọn món, hệ thống không hỗ trợ tư vấn lựa chọn dựa trên tình trạng thực tế của nhà hàng như món còn phục vụ, lượng đơn bếp đang xử lý hay những món cả bàn đã gọi.

Servio được phát triển theo hướng đồng bộ đơn hàng qua WebSocket/STOMP, bổ sung ACK, RESUME và REPLAY để khôi phục sự kiện sau khi mất kết nối. Hệ thống sử dụng Idempotency-Key để hạn chế tạo đơn trùng và Optimistic Locking để kiểm soát cập nhật đồng thời. Nhóm đánh giá các cơ chế này thông qua thực nghiệm về độ trễ, khả năng khôi phục sự kiện và tính nhất quán của dữ liệu.

Ngoài ra, Servio bổ sung một điểm mới so với các hệ thống đã khảo sát: trợ lý AI gợi ý món theo ngữ cảnh thời gian thực. Mỗi câu hỏi của khách được máy chủ ghép cùng trạng thái hiện tại của nhà hàng trước khi gửi tới mô hình ngôn ngữ Gemini; mọi gợi ý trả về đều được máy chủ kiểm tra lại trước khi hiển thị.

# III. PHƯƠNG PHÁP NGHIÊN CỨU

## 3.1. Xác định bài toán

Servio được xây dựng cho quy trình gọi món tại một nhà hàng. Khách hàng quét mã QR tại bàn, lựa chọn món và gửi đơn hàng đến máy chủ. Bộ phận bếp cần nhận được đơn mới để bắt đầu chế biến, trong khi khách hàng và thu ngân cần theo dõi những thay đổi liên quan đến đơn hàng và phiên bàn

Bài toán chính là đồng bộ thông tin giữa các thiết bị khi hệ thống có nhiều yêu cầu đồng thời và kết nối mạng không ổn định. Nhóm xác định bốn yêu cầu cần giải quyết:

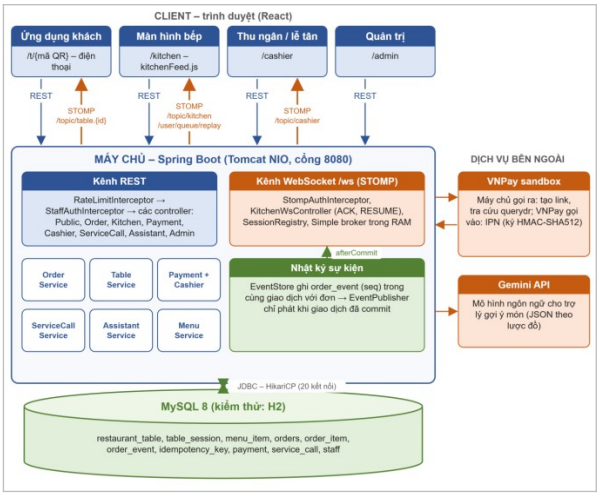
- Độ trễ cập nhật: Đơn hàng và trạng thái mới cần được chuyển đến thiết bị liên quan mà không phải chờ chu kỳ truy vấn định kỳ.
- Khôi phục sau mất kết nối: Thiết bị bếp phải có khả năng nhận lại các sự kiện chưa được tiếp nhận khi kết nối bị gián đoạn.
- Xử lý yêu cầu lặp: Một thao tác đặt món được gửi lại không được tạo thêm đơn hàng ngoài ý muốn.
- Kiểm soát cập nhật đồng thời: Hai yêu cầu thay đổi cùng một bản ghi không được âm thầm ghi đè kết quả của nhau.

Đầu vào của hệ thống gồm yêu cầu đặt món, thay đổi trạng thái món ăn và các thao tác liên quan đến phiên bàn. Đầu ra là dữ liệu nghiệp vụ đã được lưu trữ cùng những sự kiện cập nhật gửi đến các thiết bị có quyền truy cập.

Phạm vi nghiên cứu tập trung vào giao tiếp Client–Server trong một nhà hàng. Nhóm chưa triển khai hệ thống trên nhiều máy chủ hoặc đánh giá hoạt động trong mạng phân tán.

## 3.2. Kiến trúc hệ thống đề xuất

Servio sử dụng kiến trúc Client–Server với ba thành phần chính: giao diện web React, máy chủ Spring Boot và cơ sở dữ liệu MySQL. Các client giao tiếp với máy chủ thông qua HTTP/REST và WebSocket/STOMP. Hệ thống còn kết nối với VNPay để xử lý thanh toán trực tuyến và Gemini API để hỗ trợ gợi ý món ăn.




**Hình 3.1.** Kiến trúc tổng thể của hệ thống QR Order

Các client được tổ chức theo vai trò sử dụng:

- Khách hàng: Quét mã QR, tham gia phiên bàn, xem thực đơn, đặt món và theo dõi trạng thái đơn hàng.
- Bếp: Tiếp nhận đơn mới, xác nhận đơn và cập nhật tiến độ chế biến.
- Thu ngân: Theo dõi tình trạng bàn, xử lý yêu cầu phục vụ, xác nhận thanh toán và kết thúc phiên bàn.
- Quản trị viên: Quản lý bàn, thực đơn và cấu hình hệ thống.

Máy chủ chịu trách nhiệm kiểm tra quyền truy cập, xử lý nghiệp vụ, cập nhật cơ sở dữ liệu và phân phối sự kiện. Client không tự quyết định giá món ăn, tồn kho hoặc trạng thái thanh toán. Những thông tin này được xác định tại máy chủ nhằm tránh sai lệch giữa các thiết bị.

## 3.3. Thiết kế giao tiếp Client và Server

### 3.3.1. Phân chia kênh giao tiếp

Servio sử dụng hai kênh giao tiếp với nhiệm vụ khác nhau.

HTTP/REST tiếp nhận các yêu cầu nghiệp vụ như tham gia phiên bàn, tạo đơn hàng, cập nhật trạng thái và xác nhận thanh toán. Mỗi yêu cầu nhận được phản hồi từ máy chủ, bao gồm kết quả xử lý hoặc mã lỗi tương ứng.

WebSocket/STOMP được sử dụng để gửi sự kiện khi dữ liệu thay đổi. Client duy trì kết nối WebSocket và đăng ký các đích STOMP phù hợp với vai trò. Sau khi một thao tác nghiệp vụ được lưu thành công, máy chủ phát thông báo đến những thiết bị liên quan.

Cách phân chia này giúp các lệnh nghiệp vụ có phản hồi rõ ràng, trong khi những thay đổi của đơn hàng có thể được thông báo mà không cần client liên tục truy vấn.

### 3.3.2. Tổ chức các kênh STOMP

Bảng 3.1. Các đích STOMP chính của Servio

| Đích | Hướng | Nội dung | Ai được dùng |
|---|---|---|---|
| /topic/kitchen | Máy chủ → bếp | ORDER_CREATED, ITEM_STATUS_CHANGED, ORDER_CANCELLED, ORDER_UPDATED (có seq) | Nhân viên |
| /topic/table.{id} | Máy chủ → các máy của bàn | Sự kiện đơn của bàn, PAYMENT_UPDATED, SERVICE_CALL_UPDATE, TABLE_SESSION_ENDED | Khách của đúng bàn đó; nhân viên |
| /topic/cashier | Máy chủ → quầy | TABLE_STATE_CHANGED, PAYMENT_UPDATED, SERVICE_CALL_UPDATED | Thu ngân, quản trị |
| /topic/menu | Máy chủ → mọi client | MENU_UPDATED (món hết, đổi giá) | Tất cả |
| /app/ack, /app/resume | Bếp → máy chủ | {seq}, {lastSeq} | Nhân viên |
| /user/queue/replay | Máy chủ → một phiên bếp | REPLAY_BATCH {events, latestSeq, hasMore} | Nhân viên |

Máy chủ kiểm tra quyền truy cập khi client kết nối và đăng ký kênh. Khách hàng chỉ được theo dõi dữ liệu của phiên bàn hợp lệ, còn các kênh nghiệp vụ dành cho bếp và thu ngân yêu cầu quyền nhân viên tương ứng.

## 3.4. Cơ chế giao nhận sự kiện tin cậy

### 3.4.1. Đánh số và lưu trữ sự kiện

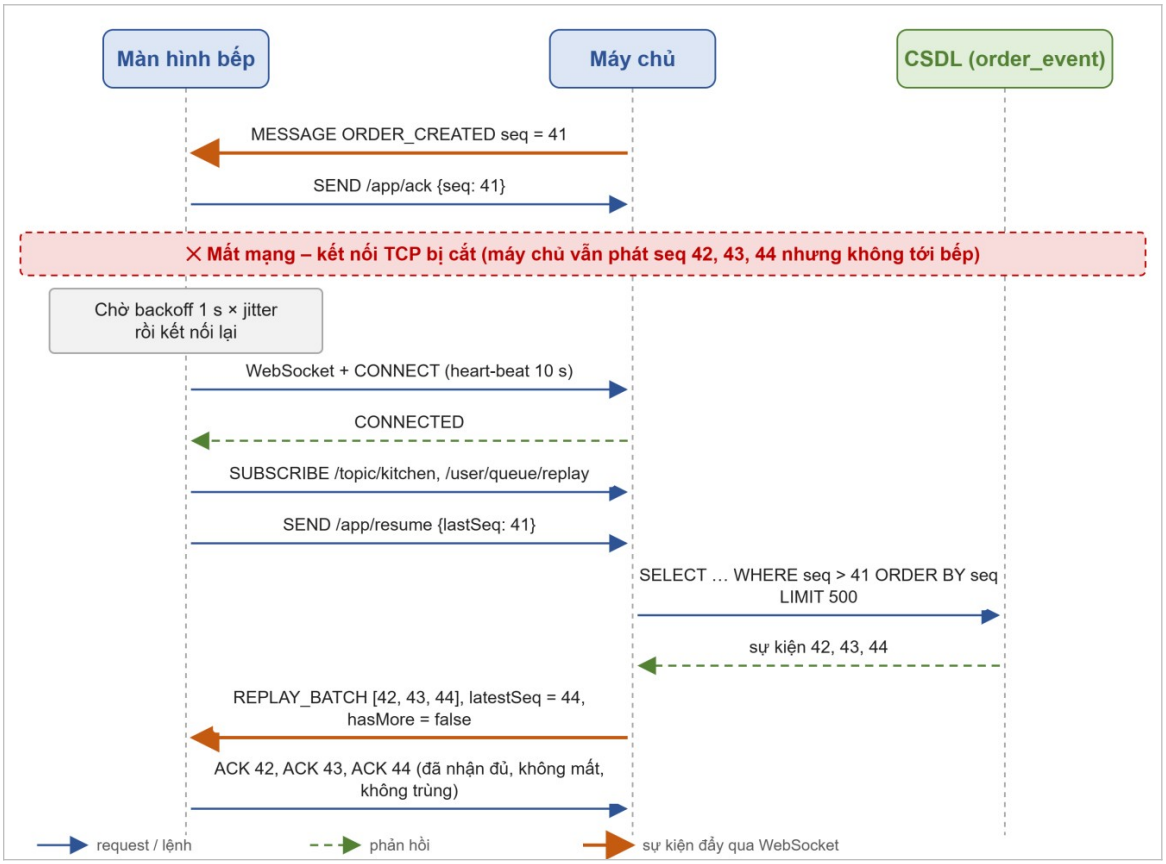
Mỗi sự kiện đơn hàng gửi đến bếp được gắn số thứ tự seq, mã định danh sự kiện, loại sự kiện, thời gian tạo và dữ liệu nghiệp vụ. Máy chủ lưu nhật ký sự kiện trong cùng giao dịch cơ sở dữ liệu với thao tác làm phát sinh sự kiện.

Sự kiện chỉ được phát đến client sau khi giao dịch hoàn tất thành công. Cách xử lý này tránh trường hợp client nhận được thông báo về một thay đổi chưa được lưu vào cơ sở dữ liệu.

### 3.4.2. Xác nhận và phát lại sự kiện

Client bếp theo dõi số thứ tự sự kiện đã tiếp nhận. Khi nhận sự kiện hợp lệ, client gửi thông điệp ACK đến máy chủ. Nếu phát hiện thiếu số thứ tự hoặc kết nối bị ngắt, client sử dụng RESUME để yêu cầu đồng bộ từ vị trí đã ghi nhận.

Máy chủ truy vấn nhật ký sự kiện và gửi các sự kiện còn thiếu qua thông điệp REPLAY_BATCH. Theo thiết kế của nhóm, mỗi lô phát lại có tối đa 500 sự kiện. Client kiểm tra số thứ tự để bỏ qua những sự kiện đã xử lý và tiếp tục yêu cầu dữ liệu nếu vẫn còn sự kiện chưa nhận.




**Hình 3.4.** Sơ đồ tuần tự ACK, RESUME và REPLAY

Cơ chế này được triển khai cho luồng giao đơn đến bếp, nơi việc bỏ lỡ thông báo có thể ảnh hưởng trực tiếp đến quá trình tiếp nhận đơn hàng.

## 3.5. Kiểm soát yêu cầu lặp lại và cập nhật đồng thời

### 3.5.1. Xử lý yêu cầu đặt món trùng lặp

Khi khách hàng gửi yêu cầu đặt món, client tạo một Idempotency-Key và gửi kèm yêu cầu HTTP. Nếu chưa nhận được phản hồi do mất mạng, client có thể gửi lại yêu cầu với cùng khóa.

Máy chủ sử dụng khóa này để nhận biết thao tác đã được xử lý. Nếu yêu cầu tương ứng đã tạo đơn thành công, server trả lại kết quả của đơn hàng trước đó thay vì tạo thêm đơn mới.

Client lưu yêu cầu chưa gửi thành công trong localStorage để có thể thử lại khi kết nối được khôi phục. Việc sử dụng lại cùng khóa giúp phân biệt thao tác gửi lại với một lần đặt món mới.

### 3.5.2. Kiểm soát cập nhật trạng thái đồng thời

Mỗi bản ghi cần kiểm soát cập nhật được gắn thông tin phiên bản (version). Khi nhân viên bếp thay đổi trạng thái món ăn, client gửi kèm phiên bản đang hiển thị.

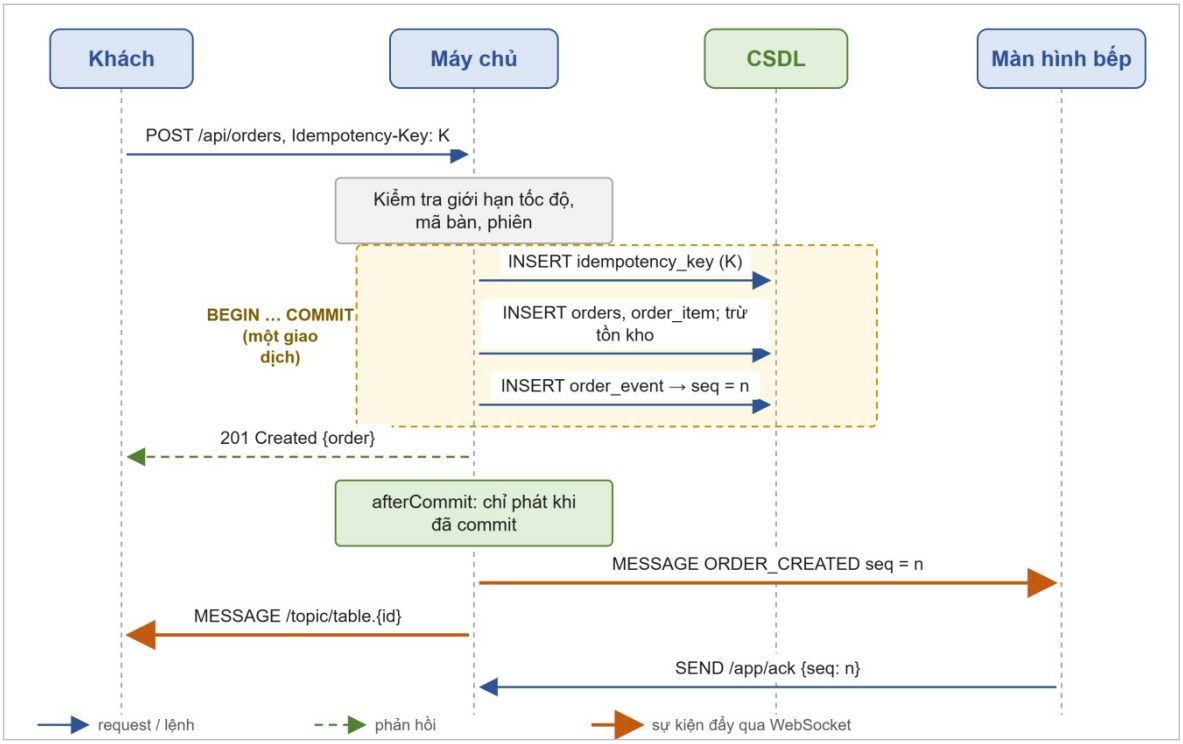
Máy chủ chỉ chấp nhận cập nhật nếu phiên bản trong yêu cầu khớp với phiên bản hiện tại của bản ghi. Nếu một thiết bị khác đã cập nhật trước, yêu cầu đến sau không được ghi đè và máy chủ trả về lỗi HTTP 409 Conflict.

Client nhận lỗi sẽ tải lại dữ liệu mới nhất trước khi thực hiện thao tác tiếp theo.

## 3.6. Quy trình xử lý đơn hàng

Quy trình chính của Servio được thực hiện như sau:

1. Khách hàng quét mã QR, truy cập giao diện đặt món và tham gia phiên bàn.
2. Khách hàng chọn món và gửi yêu cầu đặt món qua HTTP/REST cùng Idempotency-Key.
3. Máy chủ kiểm tra phiên bàn, món ăn, tồn kho và tính hợp lệ của yêu cầu.
4. Nếu yêu cầu hợp lệ, máy chủ tạo đơn hàng, cập nhật dữ liệu và ghi sự kiện trong giao dịch cơ sở dữ liệu.
5. Sau khi giao dịch hoàn tất, máy chủ phát sự kiện đơn mới đến /topic/kitchen.
6. Client bếp tiếp nhận sự kiện, cập nhật danh sách đơn và gửi ACK.
7. Khi bếp thay đổi trạng thái món, máy chủ kiểm tra phiên bản dữ liệu và gửi thông báo đến các thiết bị liên quan.




**Hình 3.5.** Sơ đồ tuần tự đặt món và giao tới bếp

Trong trường hợp mất kết nối, client bếp sử dụng cơ chế RESUME và REPLAY để nhận lại những sự kiện còn thiếu. Nếu khách hàng gửi lặp yêu cầu đặt món hoặc nhiều nhân viên cập nhật cùng một trạng thái, máy chủ áp dụng các cơ chế kiểm soát đã trình bày ở mục 3.5.

## 3.7. Trợ lý AI gợi ý món theo ngữ cảnh thời gian thực

Ngoài đồng bộ đơn hàng, Servio tích hợp một trợ lý AI giúp khách hàng chọn món bằng ngôn ngữ tự nhiên, ví dụ "món nào ra nhanh cho 3 người, không cay". Khác với chatbot chỉ biết thực đơn in sẵn, trợ lý được cung cấp trạng thái thời gian thực của nhà hàng tại thời điểm khách hỏi. Luồng xử lý một câu hỏi gồm các bước sau:

- Tiếp nhận yêu cầu: Client gửi tối đa 10 tin nhắn gần nhất của cuộc trò chuyện qua HTTP POST /api/assistant/chat kèm mã bàn và mã phiên; máy chủ xác thực phiên bàn trước khi xử lý.
- Xây dựng ngữ cảnh thời gian thực: Máy chủ đọc thực đơn còn bán kèm tồn kho, số suất từng món đang chờ hoặc đang được bếp xử lý và các món cả bàn đã gọi trong phiên, rồi đưa vào lời nhắc (prompt) gửi tới Gemini API.
- Kiểm chứng kết quả: Mô hình trả về JSON gồm câu trả lời và tối đa 4 gợi ý. Máy chủ chỉ giữ những gợi ý có mã món tồn tại, còn bán, đủ tồn kho và số lượng hợp lệ; tên và giá luôn lấy từ thực đơn của máy chủ thay vì từ mô hình, nhằm loại bỏ thông tin sai do mô hình tự sinh (hallucination).
- Thêm vào giỏ: Khách bấm thêm món được gợi ý vào giỏ và tự gửi đơn. Trợ lý không có quyền ghi dữ liệu, nên mọi đơn hàng vẫn đi qua cơ chế Idempotency-Key và kiểm tra tồn kho như luồng đặt món thông thường.

Do Gemini là dịch vụ bên ngoài được truy cập qua Internet, máy chủ áp dụng các cơ chế bảo vệ để sự cố của dịch vụ này không ảnh hưởng tới trải nghiệm gọi món:

- Giới hạn tần suất: Mỗi bàn được hỏi tối đa 5 câu liên tiếp và hồi 6 câu mỗi phút (token bucket), tránh vượt hạn mức gọi API.
- Giới hạn thời gian chờ: Mỗi lần gọi Gemini có timeout 15 giây.
- Ngắt mạch (circuit breaker): Khi Gemini báo hết lượt (HTTP 429) hoặc lỗi liên tiếp vượt ngưỡng 3 lần, máy chủ tạm ngừng gọi trong khoảng Retry-After hoặc 60 giây.
- Gợi ý dự phòng: Trong thời gian tạm ngừng hoặc khi chưa cấu hình khóa API, máy chủ trả gợi ý tính bằng quy tắc từ cùng dữ liệu thời gian thực, nên khách vẫn luôn nhận được phản hồi.

Nhờ được cung cấp trạng thái thực tế, trợ lý không đề xuất món đã hết và có cơ sở cân nhắc tải của bếp cũng như những món cả bàn đã gọi. Giao diện trợ lý được minh họa tại Hình A.4 (Phụ lục A).

## 3.8. Phương pháp đánh giá

Nhóm xây dựng các kịch bản kiểm thử để đánh giá riêng từng yêu cầu kỹ thuật:

- So sánh độ trễ tiếp nhận đơn giữa HTTP polling và WebSocket/STOMP.
- Ngắt kết nối client bếp trong quá trình tạo đơn để kiểm tra khả năng khôi phục sự kiện
- Gửi lặp yêu cầu đặt món để đánh giá cơ chế Idempotency-Key
- Cho nhiều client cập nhật đồng thời để kiểm tra khả năng phát hiện xung đột.
- Đánh giá khả năng xử lý khi số lượng client tăng và khi xuất hiện nhiều yêu cầu liên tiếp.

Các phép thử sử dụng môi trường Spring Boot với cơ sở dữ liệu H2 chạy trong bộ nhớ để thuận tiện tạo dữ liệu và lặp lại kịch bản. Kết quả đo, số lượng mẫu và nhận xét đối với từng phép thử được trình bày tại Phần IV.

# IV. THỰC NGHIỆM VÀ KẾT QUẢ

## 4.1. Môi trường và phương pháp thực nghiệm

Các thí nghiệm được thực hiện trên hệ thống Servio với máy chủ Spring Boot 3.3.5, Java 17 và cơ sở dữ liệu H2 chạy trong bộ nhớ ở chế độ tương thích MySQL. Nhóm sử dụng JUnit 5 và Spring Boot Test để kiểm thử tích hợp, kết hợp hai chương trình Node.js e2e-test.js và experiment.js để mô phỏng hoạt động của khách hàng, bếp và thu ngân.

Các thí nghiệm đối chứng được triển khai trên cùng hệ thống, thay đổi cơ chế xử lý giữa phương án cơ sở (Baseline) và phương án đề xuất. Nhóm đánh giá sáu khía cạnh: độ trễ giao đơn, khả năng khôi phục sau mất kết nối, xử lý yêu cầu trùng, kiểm soát cập nhật đồng thời, thông lượng và khả năng giới hạn yêu cầu.

## 4.2. Kết quả kiểm thử chức năng

Nhóm thực hiện kiểm thử tự động đối với quy trình đặt món, giao đơn, cập nhật trạng thái, thanh toán, phân quyền và xử lý các tình huống bất thường. Kết quả chạy ngày 25/10/2026 được tổng hợp trong Bảng 4.1.

**Bảng 4.1.** Kết quả kiểm thử tự động

| **Phạm vi kiểm thử** | **Số ca** | **Đạt** | **Lỗi** |
|---|---|---|---|
| OrderFlowIntegrationTest | 18 | 18 | 0 |
| AssistantIntegrationTest | 5 | 5 | 0 |
| OrderItemStatusTest và TokenBucketTest | 6 | 6 | 0 |
| End-to-End (19 nhóm kịch bản) | 212 | 212 | 0 |
| Tổng cộng | 241 | 241 | 0 |

Toàn bộ 29 ca kiểm thử tích hợp và 212 ca kiểm thử End-to-End đều đạt trong đợt chạy được ghi nhận. Các tình huống được kiểm tra bao gồm gửi lặp đơn hàng, nhiều thiết bị cập nhật cùng lúc, mất kết nối bếp, truy cập sai quyền và xử lý phản hồi thanh toán.

## 4.3. Phương pháp đo đạc thực nghiệm đối chứng (E1 – E6)

Thực nghiệm E1 (Độ trễ giao đơn): Đo phân phối độ trễ từ lúc đặt món tới lúc bếp nhận (trung vị p50 và phân vị p95) giữa chế độ POLLING (bếp hỏi định kỳ 3 giây/lần) và RELIABLE_PUSH (WebSocket/STOMP).

Thực nghiệm E2 (Độ tin cậy khi rớt mạng): Khách đặt 200 đơn liên tiếp, giữa chừng ngắt kết nối bếp trong 3 giây rồi nối lại; đếm số đơn bị mất giữa NAIVE_PUSH (đẩy không ACK/RESUME) và RELIABLE_PUSH.

Thực nghiệm E3 (Chống trùng) & E4 (Chống ghi đè): Ở E3, giả lập 34% request đặt món bị gửi lặp để đo tỷ lệ đơn trùng khi tắt/bật Idempotency-Key. Ở E4, cho 10 client bếp cùng cập nhật trạng thái qua 20 vòng lặp để đếm số lần mất cập nhật (Lost Update) khi tắt/bật khoá lạc quan version.

Thực nghiệm E5 (Thông lượng tải cao) & E6 (Chống spam): Ở E5, tăng tải từ 10 lên 50 và 100 client đặt món đồng thời để đo thông lượng (đơn/giây) và độ trễ p95. Ở E6, phát 45.708 request spam liên tiếp để đo số request lọt vào tầng nghiệp vụ và % CPU máy chủ (qua Actuator) khi tắt/bật Token Bucket.

### 4.4. Kết quả thực nghiệm đối chứng

Kết quả thực nghiệm E1–E6 được tổng hợp trong Bảng 4.2, so sánh các chỉ số giữa phương án cơ sở (Baseline) và phương án đề xuất của Servio.

Bảng 4.2. Kết quả thực nghiệm đối chứng E1 – E4 và E6

| Chỉ số đo lường | Baseline | Servio | Thực nghiệm |
|---|---|---|---|
| Độ trễ giao đơn trung vị (p50) | 1.341,68 ms | 7,71 ms | E1 |
| Độ trễ giao đơn phân vị p95 | 2.809,83 ms | 11,52 ms | E1 |
| Số sự kiện bị bỏ lỡ / 200 đơn | 31 | 0 | E2 |
| Tỷ lệ đơn trùng khi 34% yêu cầu được gửi lặp | 34% | 0% | E3 |
| Số lần Lost Update / 20 vòng × 10 client | 134 | 0 | E4 |
| Số yêu cầu spam đến tầng nghiệp vụ / 45.708 yêu cầu | 45.708 | 6 | E6 |
| CPU máy chủ trung bình khi bị spam | 14,00% | 4,38% | E6 |

Ở E5, hệ thống đạt thông lượng 2.536 đơn/giây với 10 client và khoảng 3.200 đơn/giây với 50–100 client, không ghi nhận lỗi.

## V. THẢO LUẬN

### 5.1. Đánh giá hiệu quả đồng bộ thời gian thực và khôi phục sự kiện

Số liệu thực nghiệm từ kịch bản E1 đến E6 và kết quả kiểm thử tự động (29/29 ca tích hợp JUnit 5, 212/212 ca End-to-End) cho thấy kiến trúc kết hợp HTTP/REST và

WebSocket/STOMP giải quyết trực tiếp bài toán đồng bộ đơn hàng tại nhà hàng. Ở kịch bản E1, việc duy trì kết nối hai chiều lâu dài qua WebSocket/STOMP giúp giảm độ trễ trung vị (p50) khi chuyển đơn từ khách đến bếp từ 1.341,68 ms (với HTTP polling chu kỳ 3 giây) xuống còn 7,71 ms (nhanh hơn khoảng 174 lần), đồng thời giảm độ trễ ở phân vị p95 từ 2.809,83 ms xuống 11,52 ms. Sự cải thiện này là do với HTTP polling, thiết bị bếp luôn phải chịu độ trễ chờ trung bình bằng nửa chu kỳ hỏi và tạo ra nhiều yêu cầu dư thừa ngay cả khi không có đơn mới; ngược lại, với STOMP, máy chủ chủ động đẩy thông điệp tới đích /topic/kitchen ngay sau khi giao dịch cơ sở dữ liệu hoàn tất (afterCommit).

Bên cạnh tốc độ phản hồi, kịch bản E2 làm rõ sự khác biệt giữa cách đẩy thông báo thông thường (NAIVE_PUSH) và giao thức giao nhận sự kiện tin cậy (RELIABLE_PUSH) của Servio. Khi màn hình bếp bị ngắt kết nối mạng trong 3 giây giữa lúc khách hàng gửi liên tiếp 200 đơn, phương án đẩy thông thường làm thất lạc 31 đơn hàng (chiếm 15,5%) do các khung tin phát ra trong lúc kết nối TCP bị đứt không được lưu vết hay truyền lại. Trong khi đó, nhờ gắn số thứ tự toàn cục seq cho từng bản ghi trong bảng nhật ký sự kiện (order_event) cùng quy trình xác nhận ACK và yêu cầu đồng bộ lại qua cặp bản tin RESUME / REPLAY_BATCH (tối đa 500 sự kiện mỗi lô), client bếp phát hiện chính xác khoảng trống số thứ tự và khôi phục đầy đủ toàn bộ 200/200 đơn hàng mà không xảy ra mất mát hay trùng lặp.

## 5.2. Đánh giá khả năng kiểm soát tương tranh, yêu cầu lặp và chống quá tải

Trong môi trường mạng không dây tại nhà hàng, hiện tượng gửi lặp yêu cầu và thao tác đồng thời diễn ra thường xuyên. Kết quả kịch bản E3 cho thấy khi 34% yêu cầu đặt món bị gửi lặp do mạng chập chờn, cấu hình cơ sở (Baseline) tạo ra đúng 34% đơn hàng trùng trong cơ sở dữ liệu. Khi áp dụng cơ chế Idempotency-Key kết hợp lưu tạm yêu cầu tại localStorage phía trình duyệt và đối chiếu mã băm tại máy chủ, tỷ lệ tạo đơn trùng giảm xuống 0%: các yêu cầu gửi lại với cùng khóa đều được máy chủ nhận diện và trả về kết quả của đơn hàng đã tạo trước đó. Tương tự, ở kịch bản E4 với 10 client bếp cùng cập nhật trạng thái trên một tập món qua 20 vòng lặp, cơ chế khóa lạc quan (Optimistic Locking) kiểm tra cột version trong câu lệnh cập nhật đã ngăn chặn toàn bộ 134 trường hợp ghi đè mất dữ liệu (Lost Update), trả về mã lỗi HTTP 409 Conflict để thiết bị tải lại trạng thái mới nhất.

Về khả năng chịu tải và bảo vệ tài nguyên, kịch bản E5 ghi nhận hệ thống đạt thông lượng 2.536 đơn/giây với 10 client đồng thời và tăng lên khoảng 3.200 đơn/giây ở mức 50–100 client mà không phát sinh lỗi. Ở kịch bản chống spam E6, khi một nguồn phát liên tiếp 45.708 yêu cầu bất thường, bộ giới hạn tốc độ Token Bucket đã chặn ngay tại tầng lọc HTTP 45.702 yêu cầu (chỉ cho phép 6 yêu cầu hợp lệ đầu tiên đi vào tầng xử lý nghiệp vụ), qua đó giảm mức tiêu thụ CPU trung bình của máy chủ từ 14,00% xuống còn 4,38% và giữ cho các bàn khách khác hoạt động ổn định.

## 5.3. Đối chiếu với các hệ thống liên quan và hạn chế của đề tài

Khi đặt kết quả của Servio bên cạnh các hệ thống mã nguồn mở đã khảo sát ở Phần II, có thể thấy rõ phạm vi đóng góp kỹ thuật của đề tài. Khác với dự án QR Menu [1] chỉ

dừng ở việc hiển thị thực đơn tĩnh và dự án Restaurant Live Ordering [3] dùng WebSocket nhưng chưa có cơ chế xử lý mất kết nối hay kiểm soát xung đột cập nhật, Servio giải quyết đồng bộ cả bốn yêu cầu từ độ trễ, khôi phục sự kiện đến chống trùng và chống ghi đè. So với dự án QR Ordering [2] (vốn khôi phục trạng thái sau khi mất mạng bằng cách gọi REST tải lại toàn bộ dữ liệu hiện tại), Servio tiến thêm một bước ở tầng giao thức khi thiết kế cơ chế phát bù từng sự kiện nghiệp vụ bị bỏ lỡ theo đúng thứ tự seq thông qua ACK, RESUME và REPLAY_BATCH, đồng thời bổ sung thực nghiệm định lượng đối chứng và tích hợp trợ lý AI gợi ý món ăn theo ngữ cảnh thực đơn.

Tuy nhiên, nhìn nhận một cách khách quan, hệ thống và quá trình thực nghiệm hiện tại vẫn còn một số hạn chế cần lưu ý:

- **Hạn chế về môi trường đo đạc thực nghiệm:** Các bài đo đối chứng E1–E6 hiện được thực hiện với cơ sở dữ liệu H2 chạy trong bộ nhớ (RAM) trên cùng một máy vật lý và mỗi cấu hình được đo một lượt để thuận tiện tái lập kịch bản. Nhóm chưa tiến hành đo lặp lại nhiều lần trên hệ quản trị MySQL 8 độc lập dưới điều kiện mạng Wi-Fi thực tế có suy hao và nhiễu sóng tại nhà hàng.
- **Hạn chế về kiến trúc triển khai và phạm vi nhật ký sự kiện:** Hệ thống hiện vận hành trên một máy chủ đơn lẻ với bộ chuyển tiếp thông điệp (Simple Broker) nằm trong bộ nhớ RAM, chưa hỗ trợ đồng bộ sự kiện xuyên suốt giữa cụm nhiều máy chủ. Ngoài ra, cơ chế đánh số thứ tự seq và phát lại sự kiện mới được áp dụng cho luồng đơn hàng của bếp (/topic/kitchen), chưa mở rộng cho các sự kiện thanh toán và gọi phục vụ tại quầy thu ngân.
- **Hạn chế về nghiệp vụ quản lý và đánh giá trợ lý AI:** Một số nghiệp vụ quản lý bán hàng (POS) nâng cao tại nhà hàng như tách hoặc gộp hóa đơn, chuyển bàn, áp dụng mã giảm giá, tính thuế VAT và thống kê doanh thu chi tiết chưa được hoàn thiện. Bên cạnh đó, nhóm chưa thực hiện khảo sát định lượng về mức độ giảm tải thao tác cho nhân viên phục vụ cũng như chưa đo lường tỷ lệ gợi ý sát ngân sách của trợ lý AI trong điều kiện phục vụ thực tế.

# VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 6.1. Kết luận

Đề tài đã hoàn thành mục tiêu nghiên cứu và xây dựng thành công hệ thống gọi món trực tuyến Servio ứng dụng kiến trúc Client–Server trên nền tảng React, Spring Boot và MySQL. Thông qua việc phân tách hợp lý giữa kênh HTTP/REST cho các thao tác nghiệp vụ và kênh WebSocket/STOMP cho luồng phân phối thông báo, hệ thống cho phép khách hàng quét mã QR tại bàn để đặt món, gọi phục vụ, thanh toán trực tuyến qua VNPay, hỏi trợ lý AI và đồng bộ tức thời trạng thái đơn hàng với bộ phận bếp cũng như quầy thu ngân.

Về mặt kỹ thuật lập trình mạng, đề tài đã giải quyết trọn vẹn bốn bài toán cốt lõi trong môi trường kết nối không ổn định và nhiều người dùng thao tác song song: (1) Giảm độ trễ trung vị khi giao đơn tới bếp xuống mức 7,71 ms nhờ đẩy sự kiện qua WebSocket/STOMP ngay sau khi giao dịch cơ sở dữ liệu hoàn tất; (2) Bảo đảm không

mất thông báo đơn hàng khi thiết bị bếp rớt mạng tạm thời nhờ cơ chế đánh số thứ tự sự kiện seq kết hợp quy trình xác nhận ACK, yêu cầu tiếp tục RESUME và phát bù theo lô REPLAY_BATCH; (3) Triệt tiêu hoàn toàn hiện tượng tạo đơn hàng trùng lặp khi khách gửi lại yêu cầu nhờ khóa lũy đẳng Idempotency-Key; và (4) Ngăn chặn xung đột ghi đè dữ liệu (Lost Update) bằng khóa lạc quan (Optimistic Locking) cùng bộ giới hạn tốc độ Token Bucket bảo vệ máy chủ trước lưu lượng truy cập bất thường. Toàn bộ các cơ chế này đều đã được kiểm chứng thông qua 241 ca kiểm thử tự động và 6 kịch bản thực nghiệm đối chứng E1–E6.

## 6.2. Hướng phát triển

Để đưa hệ thống Servio từ môi trường thử nghiệm vào vận hành thực tế tại các nhà hàng có quy mô lớn, nhóm định hướng tiếp tục phát triển đề tài theo các nội dung sau:

- **Thực nghiệm trên MySQL và môi trường mạng Wi-Fi thực tế:** Tiến hành chạy lặp lại các kịch bản đo đạc E1–E6 trên hệ quản trị cơ sở dữ liệu MySQL 8 với nhiều thiết bị di động kết nối qua mạng Wi-Fi thực tế nhằm đánh giá chính xác ảnh hưởng của độ trễ truy xuất đĩa cứng và suy hao đường truyền không dây.
- **Mở rộng phạm vi khôi phục sự kiện (Event Replay):** Đưa toàn bộ các sự kiện cập nhật thanh toán (PAYMENT_UPDATED), yêu cầu gọi phục vụ (SERVICE_CALL_UPDATED) và thay đổi trạng thái bàn vào bảng nhật ký tuần tự có gắn số thứ tự seq, giúp giao diện thu ngân và khách hàng cũng có khả năng tự động phát bù từng sự kiện bị lỡ giống như màn hình bếp.
- **Nâng cấp lên kiến trúc phân tán nhiều máy chủ:** Thay thế Simple Broker trong bộ nhớ RAM bằng hệ thống STOMP Broker chuyên dụng (như RabbitMQ hoặc ActiveMQ kết hợp Redis) và bộ giới hạn tốc độ phân tán để hỗ trợ cân bằng tải giữa nhiều máy chủ Spring Boot, đồng thời cấu hình chứng chỉ bảo mật HTTPS/WSS mặc định cho cả mạng nội bộ LAN.
- **Hoàn thiện nghiệp vụ nhà hàng (POS) và đánh giá trợ lý AI:** Bổ sung các chức năng chuyển bàn, gộp bàn, tách hóa đơn thanh toán theo từng thành viên, quản lý chương trình giảm giá, thuế VAT và báo cáo thống kê doanh thu theo ca/ngày; đồng thời bổ sung bước kiểm tra ràng buộc ngân sách tại máy chủ cho trợ lý AI và tổ chức đánh giá thực nghiệm với người dùng thật tại nhà hàng.

# TÀI LIỆU THAM KHẢO

[1] jshi98, QR Code Generator and Restaurant Menu Website, GitHub, https://github.com/jshi98/qr_menu_project.

[2] Codewithaffan, QR Ordering System, GitHub, https://github.com/Codewithaffan/qr-ordering

[3] ar363, Real-Time Restaurant Table Ordering System, GitHub, https://github.com/ar363/restaurant-live-ordering

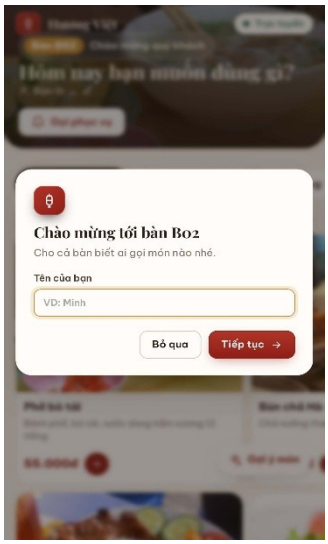
[4] J. F. Kurose, K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.

[5] R. Fielding, M. Nottingham, J. Reschke, RFC 9110 – HTTP Semantics, IETF, 2022.

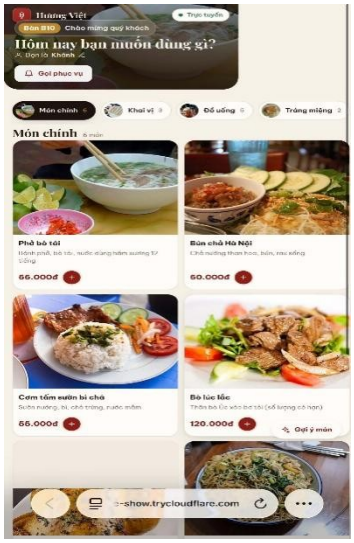
[6] I. Fette, A. Melnikov, RFC 6455 – The WebSocket Protocol, IETF, 2011.

# PHỤ LỤC

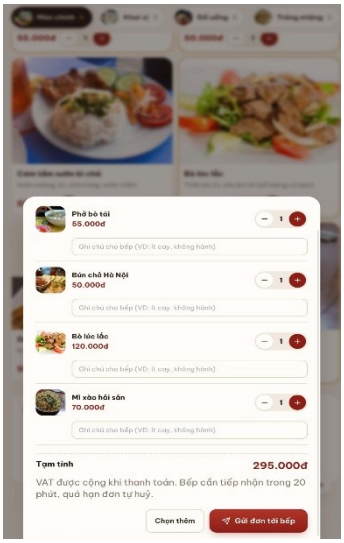
## Phụ lục A. Hình ảnh giao diện thực tế của chương trình



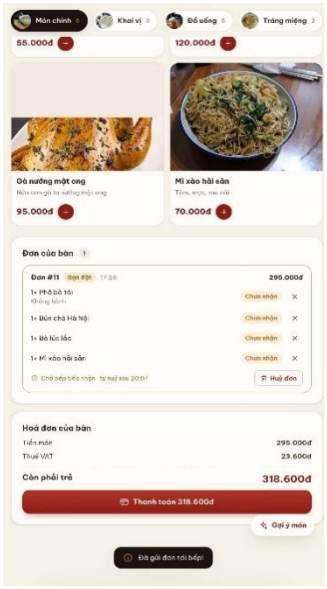

Hình A.1. Điền tên



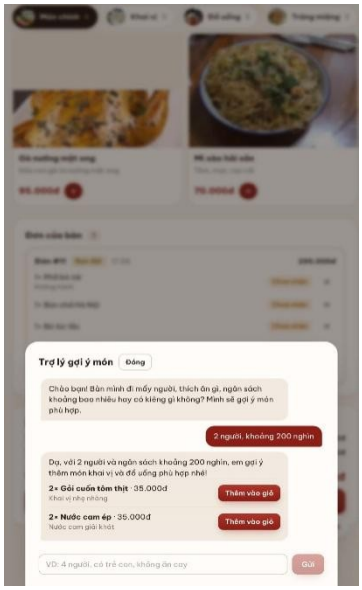

Hình A.2. Menu



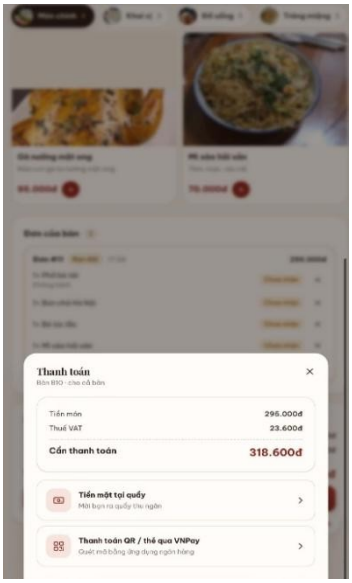

Hình A.3. Giỏ hàng



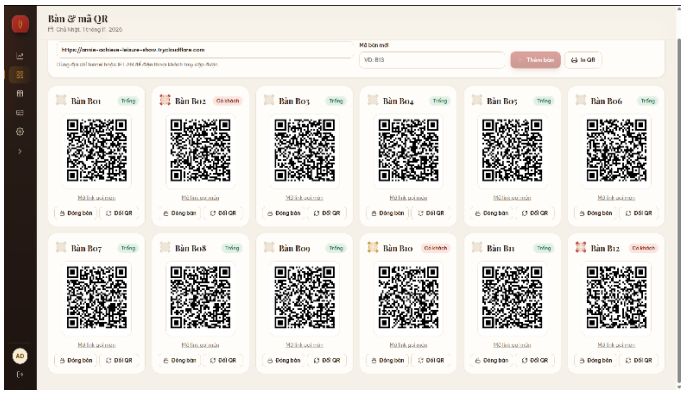

Hình A.4. Đơn của bạn



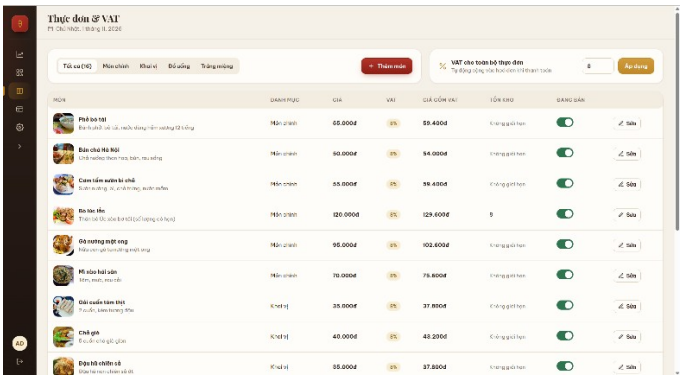

Hình A.5. Trợ lý AI



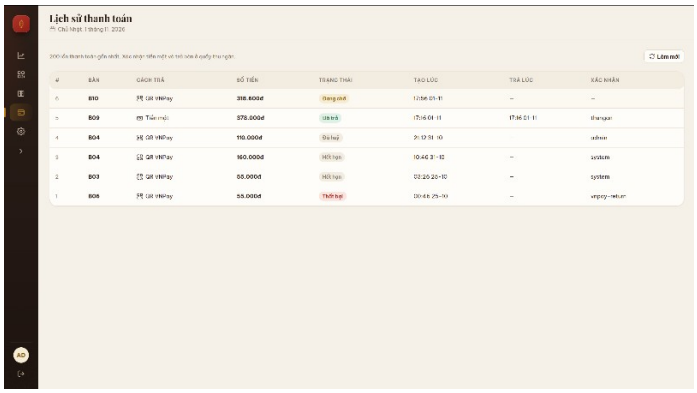

Hình A.6. Thanh toán



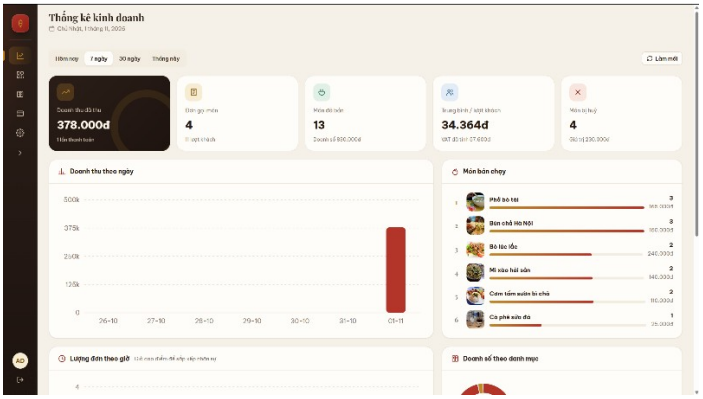

Hình A.7.Trang quản trị: Danh sách bàn và kết xuất mã QR



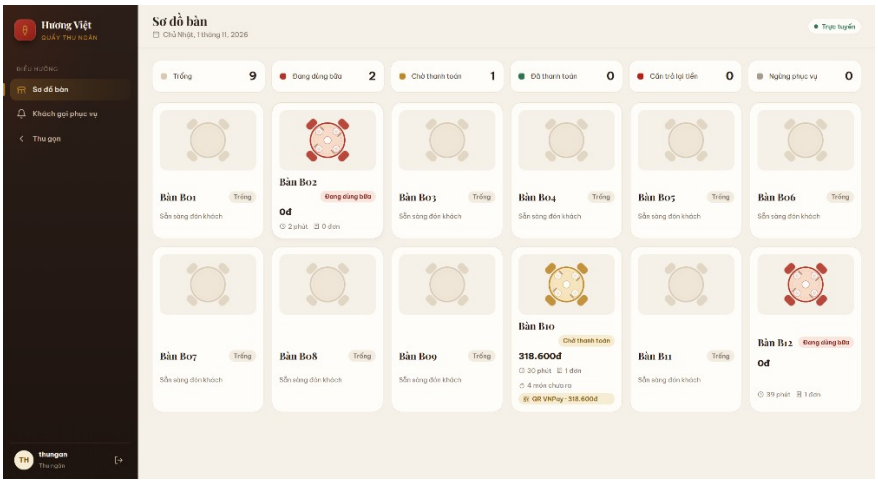

Hình A.8.Trang quản trị: Thực đơn



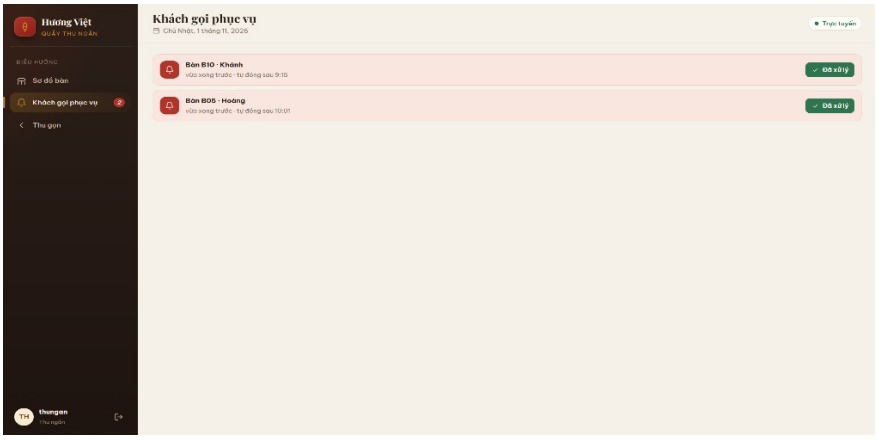

Hình A.9.Trang quản trị: Lịch sử thanh toán



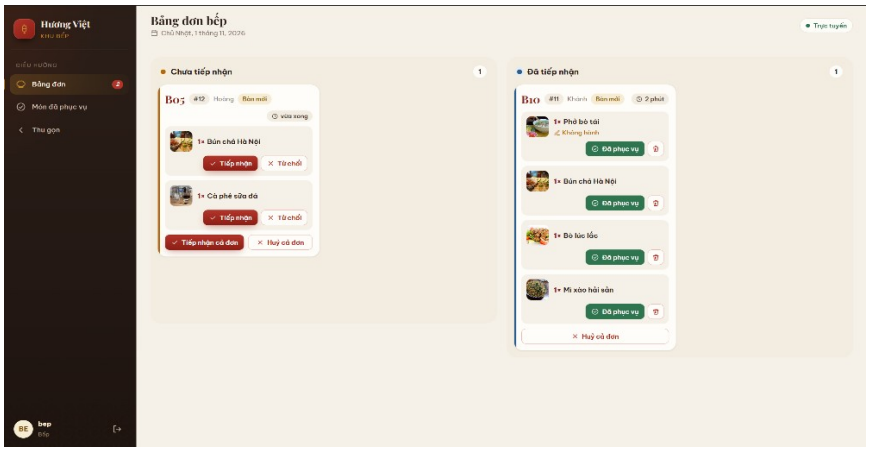

Hình A.10. Trang quản trị: Thống kê kinh doanh



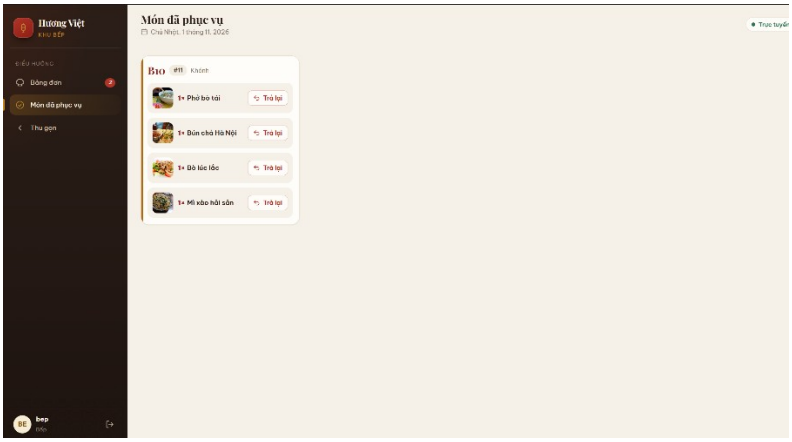

Hình A.11. Trang Thu ngân: Sơ đồ bàn




Hình A.12. Trang Thu ngân: Yêu cầu phục vụ khách hàng




Hình A.13. Trang Bếp: Bảng đơn hàng




Hình A.14. Trang bếp: Món đã phục vụ

## Phụ lục B. Hướng dẫn khởi động hệ thống, tài khoản mẫu và chạy kiểm thử

Yêu cầu môi trường: JDK 17+, MySQL 8.0 và IntelliJ IDEA (hoặc Maven); không cần cài Node.js khi chỉ chạy hệ thống vì bản build giao diện đã tích hợp sẵn.

**Lệnh lấy mã nguồn, khởi chạy máy chủ, mở ngrok và chạy kiểm thử tự động:**

```
git clone -b feat/separate_visual https://github.com/LDCin/LTM.git
# IntelliJ: mở LTM/source/server -> Run QrOrderApplication với biến môi trường DB_PASSWORD, GEMINI_API_KEY
ngrok http 8080 --url=<ten-mien>.ngrok-free.dev     # Mở truy cập từ mạng ngoài (4G) & nhận IPN VNPay
cd source/server && mvn test                        # Chạy 29 ca kiểm thử JUnit 5
cd source/loadtest && npm install && npm run e2e    # Chạy 212 ca kiểm thử End-to-End (máy chủ đang chạy)
cd source/loadtest && node run-all.js               # Chạy toàn bộ thực nghiệm đối chứng E1 - E6
```

**Bảng B.1. Địa chỉ truy cập các phân hệ và tài khoản đăng nhập mẫu**

| Phân hệ | Địa chỉ URI | Tài khoản mẫu | Ghi chú vai trò |
|---|---|---|---|
| **Trang quản trị** | /admin | admin / admin123 | Quản lý bàn, mã QR, thực đơn, cấu hình thực nghiệm |
| **Màn hình bếp** | /kitchen | bep / bep123 | Nhận đơn thời gian thực, cập nhật trạng thái & huỷ món |
| **Quầy thu ngân / lễ tân** | /cashier | thungan / thungan123 | Sơ đồ bàn, xử lý gọi phục vụ, thu tiền mặt, trả bàn |
| **Ứng dụng khách hàng** | /t/{mã QR} | Không cần tài khoản | Quét mã QR in tại trang quản trị để vào phiên bàn |